HAFTA 3 • CUMARTESİ

Transformer Mimarisi

BERT, GPT & Fine-Tuning

Self-Attention → Positional Encoding → Encoder/Decoder → BERT → GPT → Hugging Face Fine-Tuning

09:00 — 17:00 · 7 Saat · 4 Python Dosyası · 800+ Satır Kod · HuggingFace Ekosistemi

Günün Planı — Hafta 3 Cumartesi

09:00–09:30
Geçen Hafta Özeti & NLP Yol Haritası

09:30–10:45
Transformer Mimarisi: Attention is All You Need

10:45–11:00
☕ Kısa Mola

11:00–12:00
BERT: Encoder-Only, MLM, NSP, Tokenizer

12:00–13:00
GPT Ailesi: Decoder-Only, Otoregresif Modeller

13:00–14:00
🍽 Öğle Arası

14:00–15:30
Hugging Face: Tokenizer, Pipeline, Fine-Tuning

15:30–16:30
NER, Soru-Cevap, Zero-Shot Uygulamaları

16:30–17:00
Mini Proje & Kapanış

🎯 **Gün Sonu: Transformer sıfırdan kodu yazıldı + BERT fine-tuning %91+ + Çoklu NLP görevi pipeline tamamlandı**

Transformer Mimarisi — 'Attention is All You Need' (2017)

Encoder · Decoder · Self-Attention · Cross-Attention · Residual

ENCODER

- Input Embedding + Positional Encoding
- Multi-Head Self-Attention
- Add & Layer Norm
- Feed-Forward Network (FFN)
- Add & Layer Norm
- × N Encoder Katmanı (BERT=12/24)
- Output: Bağlamsal Token Temsilleri

→ Cross Att.

DECODER

- Output Embedding + Positional Encoding
- Masked Multi-Head Self-Attention
- Add & Layer Norm
- Multi-Head Cross-Attention (enc←dec)
- Add & Layer Norm
- Feed-Forward Network + Add&Norm
- Linear + Softmax → Kelime Üretimi

BERT=Yalnızca Encoder (N=12 veya 24) · GPT=Yalnızca Decoder · T5/BART=Encoder+Decoder · Orijinal Transformer (Vaswani 2017)=Encoder+Decoder

Scaled Dot-Product Attention — Temel Mekanizma

Q · K · V Matrisleri · Ölçekleme · Causal Mask

$$\text{Attention}(Q, K, V) = \text{softmax}(\, QK^T / \sqrt{d_k}\,) \cdot V$$

Q — Query (Sorgu)

`X · W_Q`

Mevcut konumun 'ne arıyorum?' sorusu. Her token kendi sorgu vektörünü üretir. Boyut: (seq_len, d_k)

K — Key (Anahtar)

`X · W_K`

Diğer konumların 'ne sunuyorum?' cevabı. Sorgu ile çarpılarak ilgi skoru üretilir. Boyut: (seq_len, d_k)

V — Value (Değer)

`X · W_V`

Gerçekten aktarılacak bilgi. Attention ağırlıkları V ile çarpılarak bağlamsal temsil üretilir. Boyut: (seq_len, d_v)

1 — **QK^T — Benzerlik Skoru**

Q matrisini K'nın transpozuyla çarp. Her (i,j) çifti için token i'nin token j'ye olan ilgisi hesaplanır. Sonuç: (seq_len, seq_len)

2 — **/ √d_k — Ölçekleme**

d_k boyutunun kareköküne bölme. Büyük boyutlarda iç çarpım çok büyür → softmax çok keskin → gradyan çok küçük. √d_k ile normalize.

3 — **+ Mask (isteğe bağlı)**

Decoder'da gelecek konumlar maskelenir (−∞). Softmax sonrası 0 olur → otoregresif üretim mümkün.

4 — **softmax() — Normalize**

Her satır için dikkat ağırlıkları [0,1] aralığına çekilir. Satır toplamı = 1. Her token diğerlerine ne kadar bakıyor?

5 — **· V — Context Vektörü**

Normalize dikkat ağırlıklarıyla V matrisinin ağırlıklı ortalaması alınır. Çıktı: her token için bağlamsal temsil.

Self-Attention'da Q=K=V=X (aynı dizi). Cross-Attention'da Q=decoder, K=V=encoder. Parametre sayısı: 3×d_model×d_k per baş.

Multi-Head Attention & Positional Encoding

h Paralel Kafa · Farklı Bağlam · Sinüzoidal PE

Multi-Head Attention

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h) \cdot W_O$$

$$\text{head}_i = \text{Attention}(QW_iQ,\ KW_iK,\ VW_iV)$$

h Paralel Kafa

BERT-base: h=12, d_k=64. Her kafa farklı ilişki boyutunu öğrenir.

Farklı Perspektifler

Kafa 1=sözdizimi, Kafa 2=anlamsal, Kafa 3=coreference gibi.

Concat + W_O

h kafa çıktısı birleştirilerek son doğrusal projeksiyondan geçer.

Parametre Verimliliği

h kafa × d_k = d_model. Parametre sayısı sabit kalır, temsil gücü artar.

```
layers.MultiHeadAttention(num_heads=8, key_dim=64,
dropout=0.1)(q, k, v)
```

Positional Encoding — Konum Bilgisi

Neden Gerekli?

Transformer, RNN'den farklı olarak sırayı doğal olarak bilmez. 'Kedi köpeği ısırdı' ile 'Köpek kediyi ısırdı' self-attention için aynı token setidir. PE konumu inject eder.

Sinüzoidal PE (orijinal)

```
PE(pos,2i)   = sin(pos / 10000^(2i/d_model))
PE(pos,2i+1) = cos(pos / 10000^(2i/d_model))
```

Öğrenilen PE
BERT/GPT: PE parametreleri eğitimde öğrenilir. Daha esnek ama uzun dizi genellemesi zayıf.

RoPE (Rotary PE)
LLaMA, GPT-NeoX. Rotasyon matrisiyle. Uzun bağlamlarda daha iyi genelleme.

ALiBi
MPT, BLOOM. Dikkat skoruna lineer bias ekler. Eğitimden uzun bağlam çıkarımı.

BERT-base: h=12, d_model=768, d_k=64, FFN=3072 · BERT-large: h=16, d_model=1024 · GPT-3: h=96, d_model=12288

Transformer Encoder Bloğu — Add&Norm & FFN

Pre-Norm vs Post-Norm · LayerNorm · Feed-Forward Network

encoder_block.py

```
class EncoderBlock(keras.layers.Layer):
    def __init__(self, d_model, n_heads, dff, dropout=0.1):
        super().__init__()
        self.mha  = layers.MultiHeadAttention(
            num_heads=n_heads,
            key_dim=d_model//n_heads,
            dropout=dropout)
        self.ffn  = keras.Sequential([
            layers.Dense(dff, activation='gelu'),
            layers.Dropout(dropout),
            layers.Dense(d_model),
        ])
        self.ln1  = layers.LayerNormalization(epsilon=1e-6)
        self.ln2  = layers.LayerNormalization(epsilon=1e-6)
        self.drop = layers.Dropout(dropout)

    def call(self, x, training=None, mask=None):
        # ── Multi-Head Self-Attention ──────────────────
        attn_out = self.mha(x, x, x,           # Q=K=V=x
                            attention_mask=mask,
                            training=training)
        x = self.ln1(x + self.drop(attn_out, training=training))
        # ── Feed-Forward Network ───────────────────────
        ffn_out = self.ffn(x, training=training)
        x = self.ln2(x + self.drop(ffn_out, training=training))
```

Add & Layer Norm (Post-Norm)

Orijinal Transformer (Vaswani 2017). x + SubLayer(x) → LayerNorm. Eğitim başlangıcında unstable olabilir.

Pre-Norm (Modern Tercih)

LayerNorm(x + SubLayer(LayerNorm(x))). GPT-2/3, LLaMA kullanır. Daha stabil eğitim, warmup gereksiz.

FFN: neden 4x genişleme?

BERT: d_model=768, dff=3072 (4x). Matris faktorizasyonu gibi. Geniş FFN = daha fazla 'bellek'. GELU aktivasyon standart.

LayerNorm vs BatchNorm

LayerNorm: örnek başına normalize (batch size bağımsız). Transformer için standart. BN sekans uzunluklarında sorunlu.

GELU(x) = x · Φ(x) (Gaussian Error Linear Unit) | BERT ve GPT-2'den beri ReLU'nun yerini aldı | SwiGLU (LLaMA): daha iyi performans

BERT — Bidirectional Encoder Representations from Transformers

MLM · NSP · WordPiece · CLS Token · Fine-Tuning Paradigması

BERT-base: 12 Encoder katmanı, 12 dikkat kafası, d_model=768, 110M parametre, çift yönlü bağlam

BERT-large: 24 katman, 16 kafa, d_model=1024, 340M parametre · Eğitim: BooksCorpus + Wikipedia (3.3B token)

Masked Language Model (MLM)

Girdi tokenların %15'i rastgele maskelenir ([MASK]). Model maskelenen tokenu tahmin eder.

80% [MASK], 10% rastgele token, 10% değişmeden kalır. Bu strateji fine-tuning ile eğitim arasındaki farkı azaltır.

```
Örnek: [CLS] Bugün hava [MASK] güzel. → çok
```

Next Sentence Prediction (NSP)

İki cümle verilir. Model 'B cümlesi gerçekten A'yı takip ediyor mu?' sorusuna yanıt verir.

50% gerçek çift, 50% rastgele çift. CLS token çıktısı 2-sınıflı sınıflandırmaya beslenir. Q&A ve NLI görevlerine yardımcı.

```
Örnek: [CLS] Güzel bir gün. [SEP] Dışarı çıktım. →
IsNext
```

BERT Fine-Tuning Görevleri

Sınıflandırma

CLS token → Dense

NER / Token Sınıf.

Her token → Dense

Soru-Cevap

Start/End span

NLI

CLS → 3-sınıf

GPT Ailesi — Decoder-Only Otoregresif Modeller

Causal LM · In-Context Learning · RLHF · Scaling Laws

BERT (Encoder-Only)

Çift yönlü bağlam. MLM ile eğitim. Sınıflandırma, NER, QA için mükemmel. Metin üretimi yapamaz.

GPT (Decoder-Only)

Tek yönlü (sol→sağ). Causal LM ile eğitim. Metin üretimi, tamamlama, dialog için. Masked attention.

GPT-1 — 2018 | 117M

Decoder-only Transformer. İlk büyük ön-eğitimli LM. Fine-tuning paradigması.

GPT-2 — 2019 | 1.5B

Zero-shot görevler. 'Tehlikeli' olduğu gerekçesiyle başlangıçta gizlendi. WebText.

GPT-3 — 2020 | 175B

Few-shot learning. In-context learning devrimleşti. API ile erişim.

InstructGPT — 2022 | 1.3B+RLHF

RLHF ile hizalama. Küçük model, daha faydalı çıktı. ChatGPT temeli.

GPT-4 — 2023 | ~1T(tahmini)

Çok modaliteli (metin+görüntü). Uzun bağlam (128K). Ekip çalışması.

LLaMA / Mistral — 2023-24 | 7B–70B

Açık ağırlıklı. Verimli mimari (GQA, SwiGLU, RoPE). LoRA uyumlu.

RLHF: Reinforcement Learning from Human Feedback — İnsan tercihiyle hizalama · Scaling Laws: parametre↑ + veri↑ = performans↑ (Chinchilla optimumu)

Hugging Face Ekosistemi — 'GitHub of AI'

transformers · datasets · tokenizers · PEFT · Inference API

huggingface_giris.py

```python
# ── 1. PIPELINE — En Hızlı Yol ──────────────────────
from transformers import pipeline

# Duygu analizi
clf = pipeline('sentiment-analysis',
               model='distilbert-base-uncased-finetuned-sst-2-english')
print(clf('Bu film harikaydı!'))
# [{'label': 'POSITIVE', 'score': 0.997}]

# ── 2. MODEL + TOKENİZER — Esnek Yol ────────────────
from transformers import AutoTokenizer, AutoModel
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model     = AutoModel.from_pretrained('bert-base-uncased')

inputs = tokenizer('Hello world!', return_tensors='pt',
                   padding=True, truncation=True, max_length=128)
outputs = model(**inputs)
cls_emb = outputs.last_hidden_state[:, 0, :]  # [CLS]

# ── 3. FINE-TUNING SÜRECİ ──────────────────────────
from transformers import AutoModelForSequenceClassification
model = AutoModelForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2)
# model.classifier → yeni başlık otomatik eklendi!
```

transformers

1000+ ön-eğitimli model. AutoModel, AutoTokenizer ile evrensel arayüz. TF, PyTorch, JAX desteği.

datasets

Standart NLP veri setleri (GLUE, SQuAD, IMDB...). Load_dataset() ile tek satır. Büyük veri Arrow formatı.

tokenizers

Rust tabanlı, çok hızlı WordPiece, BPE, SentencePiece. Offset mapping, özel token desteği.

PEFT / LoRA

Parametre-Verimli Fine-Tuning. Modelin %1'i eğitilir. LoRA: low-rank matris eklentisi. Hafıza dostu.

```
pip install transformers datasets tokenizers peft accelerate evaluate sentencepiece scikit-learn
```

```
args = TrainingArguments(output_dir='./results',
```

Fine-Tuning Stratejileri — Tam, Kısmi, LoRA

Full Fine-Tuning · Layer Freezing · LoRA · Prompt Tuning

1 Full Fine-Tuning

Tüm model parametreleri güncellenir. En yüksek performans potansiyeli. Büyük GPU gerektirir.

✓ En iyi doğruluk | ⚠ Catastrophic forgetting riski, GPU hafıza

```
for param in model.parameters():
    param.requires_grad = True
# tüm parametreler güncelleniyor
```

2 Layer Freezing (Kısmi)

Encoder katmanlarının ilk N tanesi dondurulur. Sadece üst katmanlar + başlık eğitilir.

✓ Hızlı, az hafıza | ⚠ Full FT'ye göre düşük performans

```
for layer in model.bert.encoder.layer[:8]:
    for p in layer.parameters():
        p.requires_grad = False
```

3 LoRA (Low-Rank Adaptation)

Orijinal ağırlıklar dondurulur. Her hedef matrise küçük rank-r ek matris eklenir: ΔW = A·B (r<<d).

✓ Parametre etkin, modeli bozma yok | ⚠ Hiperparametre ayarı (r, alpha, hedef katmanlar)

```
from peft import LoraConfig, get_peft_model
config = LoraConfig(r=16, lora_alpha=32,
    target_modules=['q','v'])
model = get_peft_model(model, config)
```

4 Prompt Tuning

Model ağırlıkları tamamen dondurulur. Girdi başına öğrenilmiş 'soft prompt' token eklenir.

✓ Model değişmiyor, çok ucuz | ⚠ Full FT'den zayıf, karmaşık görevlerde yetersiz

```
# Prefix-tuning örneği
prefix = nn.Parameter(torch.randn(20, d_model))
# Her batch'e prefix eklenir
```

QLoRA: 4-bit quantization + LoRA → 65B modeli tek GPU'da fine-tune · Adapter: FFN'e küçük modüller · Prefix-tuning: GPT-3 API'da prompt

Tokenizer Derinlemesine — WordPiece, BPE, SentencePiece

Tokenization · Özel Tokenlar · input_ids · attention_mask · token_type_ids

tokenizer_detay.py

```
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained('bert-base-uncased')

text = 'Transformers are amazing!'
enc  = tok(text, padding=True, truncation=True,
           max_length=128, return_tensors='pt')

# enc sözlüğü şunları içerir:
# input_ids:      [101, 19081, 2024, 6429, 999, 102]
# attention_mask: [1,   1,     1,    1,    1,   1  ]
# token_type_ids: [0,   0,     0,    0,    0,   0  ]  # BERT çift cümle

# Tokenlara bak:
tokens = tok.convert_ids_to_tokens(enc['input_ids'][0])
# ['[CLS]','transform','##ers','are','amazing','!','[SEP]']
# ##ers → alt-kelime devamı

# Batch encoding + offset mapping:
enc2 = tok(text, return_offsets_mapping=True)
# Her tokenın karakter konumunu verir → NER için kritik!
```

WordPiece (BERT)

Kelimeyi alt-kelimelere böler: 'playing' → 'play', '##ing'. Bilinmeyen kelimeler yönetilebilir. 30K vocab.

BPE — Byte-Pair Encoding (GPT)

Sık geçen karakter çiftlerini birleştirir. GPT-2: 50K vocab. Dil bağımsız. Unicode baytları.

SentencePiece (T5, LLaMA)

Dil bağımsız, boşluk özel token. Unicode karakterleri doğrudan işler. Türkçe için iyi.

Özel Tokenlar

attention_mask neden önemli?

1=gerçek token, 0=padding. Model 0 olan konumlara dikkat etmez. Batch'te farklı uzunluklu cümlelerde doğru hesaplama için şart.

token_type_ids neden önemli?

BERT çift cümle görevleri için: cümle A=0, cümle B=1. NLI, QA görevlerinde ayrımı belirler. Tek cümlede tümü 0.

Transformer Tabanlı NLP Görevleri — Karşılaştırma

Metin Sınıflandırma · NER · Soru-Cevap · Özetleme · Üretim

Görev	Önerilen Model	Mimari Başlık	Metrik	Örnek Kullanım
Metin Sınıflandırma	BERT/DistilBERT	CLS token → Dense(n_labels)	**Acc, F1**	Duygu analizi, spam, konu etiketleme
NER / Token Sınıf.	BERT-NER	Her token → Dense(n_entities)	**F1-entity**	Kişi, yer, kurum tespiti
Soru-Cevap (Extract.)	BERT-QA	Start+End pozisyon tahmini	**EM, F1**	SQuAD, bağlam içi cevap bulma
Metin Üretimi	GPT-2/T5	Otoregresif token üretimi	**Perplexity**	Hikaye, kod, dialog üretimi
Özetleme	BART/T5	Encoder-Decoder seq2seq	**ROUGE**	Haber, makale özetleme
Zero-Shot Sınıf.	MNLI modeli	Entailment skoru	**Acc**	Etiket listesiyle fine-tuningsiz sınıf

```
pipeline('text-classification') | pipeline('ner') | pipeline('question-answering') | pipeline('summarization') | pipeline('zero-shot-classification')
```

Bugünkü Uygulamalar — 4 Python Dosyası

Sırasıyla çalıştır · pip install transformers datasets peft

01 Transformer Sıfırdan

`uygulama_01_transformer_sifirdan.py`

- Positional Encoding (sinüzoidal + görsel)
- Scaled Dot-Product Attention NumPy ile
- Multi-Head Attention tf.keras.layers.Layer
- Encoder Block: MHA + FFN + Add&Norm
- IMDB sınıflandırma — LSTM vs Transformer karşılaştırma

02 BERT Fine-Tuning (IMDB)

`uygulama_02_bert_finetuning.py`

- DistilBERT/BERT AutoTokenizer + AutoModel
- 3 strateji: Feature Ext. / Kısmi / Tam FT
- HuggingFace Trainer API ile eğitim
- Öğrenme oranı ısınma + AdamW
- Precision/Recall/F1/AUC kapsamlı analiz

03 NER — Varlık Tanıma

`uygulama_03_ner.py`

- CoNLL-2003 NER veri seti
- BERT token sınıflandırma başlığı
- Offset mapping ile token→karakter hizalama
- Varlık düzeyinde F1 hesabı (seqeval)
- Tahmin görselleştirme (renkli çıktı)

04 Çoklu NLP Pipeline

`uygulama_04_nlp_pipeline.py`

- Duygu analizi + güven skoru
- NER pipeline (PER/ORG/LOC/MISC)
- Zero-shot sınıflandırma (etiket-serbest)
- Soru-Cevap (bağlam + soru → cevap)
- Metin üretimi (GPT-2 beam+sample karşılaştırma)

```
pip install transformers datasets peft evaluate seqeval scikit-learn torch numpy matplotlib
```

Hafta 3 Cumartesi Tamamlandı!

Bu günün kazanım listesi:

- ✓ Transformer mimari: Attention, MHA, PE, Add&Norm, FFN
- ✓ GPT ailesi: otoregresif, RLHF, scaling laws
- ✓ Fine-tuning: Full / Layer Freeze / LoRA / Prompt Tuning
- ✓ NLP görevleri: sınıflandırma, NER, QA, zero-shot, üretim
- ✓ BERT: MLM, NSP, CLS token, fine-tuning paradigması
- ✓ Hugging Face: pipeline, AutoModel, Trainer API
- ✓ Tokenizer: WordPiece, BPE, attention_mask, offset_mapping
- ✓ Transformer sıfırdan kod yazdık, BERT fine-tune ettik

Hafta 3 Pazar: LLM Fine-Tuning · LoRA/QLoRA · RAG · LangChain · LLM Uygulamaları